Design Linked List

Difficulty: Medium | Patterns: Linked List, Design, Sentinel Nodes | LeetCode 707

1. Problem Summary

Design a `MyLinkedList` class for a 0-indexed linked list. It must support reading by index, inserting at the head, inserting at the tail, inserting before an index, and deleting at an index. Invalid reads return `-1`; invalid mutations do nothing.

2. Constraints and Observations

- `0 <= index, val <= 1000`.
- At most `2000` total method calls.
- The problem permits either a singly or doubly linked list.
- A linked list cannot jump directly to an arbitrary index. It must follow links node by node.

The small call limit means many linked-list designs pass, but the cleanest design should reduce boundary cases and unnecessary traversal.

3. Pattern Recognition

This is a linked-list design problem. The main risk is not a hard formula; it is maintaining valid structure after many operations. The right pattern is to keep a `size` counter plus dummy boundary nodes so each operation has a simple invariant to preserve.

4. Naive Approach

A Python list would make `get` constant time, but it is not the intended linked-list implementation and middle insertions/deletions still shift elements. A more relevant naive linked-list approach is a singly linked list with only `head`. That makes `addAtHead` easy, but `addAtTail`, `get`, indexed insertion, and indexed deletion all require walking from the front.

The bottleneck is one-way traversal and repeated special cases for empty lists, head insertion, tail insertion, and deleting the first node.

5. Key Intuition

> Use dummy `head` and `tail` sentinels. Then every insertion means "put a node between two existing nodes," and every deletion means "connect the removed node's two neighbors."

With a doubly linked list, we can also start traversal from whichever end is closer to the target index.

6. Optimal Solutions Ranked

Rank	Variant	Time per operation	Space	Tradeoff
1	Singly linked list with `head` and `size`	`addAtHead O(1)`, indexed operations `O(index)`, `addAtTail O(n)`	`O(n)`	Simple, but tail append and edge cases are awkward.
2	Singly linked list with `head`, `tail`, and `size`	`addAtHead O(1)`, `addAtTail O(1)`, indexed operations `O(index)`	`O(n)`	Improves append, but arbitrary access is still one-way.
3	Doubly linked list with sentinels and `size`	Head/tail operations `O(1)`; indexed operations `O(min(index, n - index))`	`O(n)`	Best linked-list design here: fewer edge cases and shorter traversal.

The third variant is the most optimal linked-list implementation. A plain linked list cannot make arbitrary `get(index)` truly `O(1)`, because reaching an index requires following links.

7. Most Optimal Approach

Use a doubly linked list with two permanent dummy nodes:

```
empty: head <-> tail
values: head <-> 1 <-> 2 <-> 3 <-> tail
```

Store only real values between the sentinels. Keep `size` equal to the number of real nodes.

8. Algorithm

1. Create a `Node` with `val`, `prev`, and `next`.
2. Initialize `head.next = tail` and `tail.prev = head`.
3. Use `size` to reject invalid indices before traversal.

4. Find an index by walking from the closer side.
5. Insert by connecting `prev_node -> new_node -> next_node`.
6. Delete by connecting `node.prev` directly to `node.next`.

9. Visual Explanation

Sentinel nodes remove special cases: every insertion and deletion reconnects two neighbors.

Sentinel nodes remove boundary special cases: every real node always has a previous and next neighbor.

10. Dry Run

Step	Operation	List after operation	Return
1	`MyLinkedList()`	`[]`	`null`
2	`addAtHead(1)`	`[1]`	`null`
3	`addAtTail(3)`	`[1, 3]`	`null`
4	`addAtIndex(1, 2)`	`[1, 2, 3]`	`null`
5	`get(1)`	`[1, 2, 3]`	`2`
6	`deleteAtIndex(1)`	`[1, 3]`	`null`
7	`get(1)`	`[1, 3]`	`3`

11. Correctness Argument

Invariant: after every operation, the real list is exactly the sequence reached by following `next` pointers from `head.next` until `tail`, and the reverse sequence is reached by following `prev` pointers from `tail.prev` until `head`.

Initialization satisfies this invariant because there are no real nodes and the two sentinels point to each other. Insertion preserves it because the new node is placed between two adjacent existing nodes and both directions are updated. Deletion preserves it because the removed node's previous and next neighbors are connected directly to each other. The helper that finds a node moves one link per position from the closer end, so it reaches exactly the requested index. Therefore each valid operation affects the correct node or position, while invalid indices are rejected before changing the list.

12. Complexity Analysis

- `get(index)`: `O(min(index, n - 1 - index))` time, `O(1)` extra space.
- `addAtHead(val)`: `O(1)` time, `O(1)` extra space.
- `addAtTail(val)`: `O(1)` time, `O(1)` extra space.
- `addAtIndex(index, val)`: `O(min(index, n - index))` time, `O(1)` extra space beyond the new node.
- `deleteAtIndex(index)`: `O(min(index, n - 1 - index))` time, `O(1)` extra space.
- Total list storage: `O(n)`.

13. Edge Cases

- Empty list and `get(0)`: return `-1`.
- `addAtIndex(size, x)`: append to the tail.
- `addAtIndex(size + 1, x)`: do nothing.
- Deleting the only node: reconnect `head` directly to `tail`.
- Deleting the last real node: reconnect the previous real node to `tail`.

14. Final Clean Code

```
class Node:
    def __init__(self, val: int = 0):
        self.val = val
        self.prev = None
        self.next = None


class MyLinkedList:

    def __init__(self):
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head
        self.size = 0

    def _get_node(self, index: int) -> Node:
        if index < self.size // 2:
            curr = self.head.next
            for _ in range(index):
                curr = curr.next
        else:
            curr = self.tail.prev
            for _ in range(self.size - 1 - index):
                curr = curr.prev
        return curr

    def _insert_between(self, val: int, prev_node: Node, next_node: Node) -> None:
        node = Node(val)
        node.prev = prev_node
        node.next = next_node
        prev_node.next = node
        next_node.prev = node
        self.size += 1

    def _delete_node(self, node: Node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1

    def get(self, index: int) -> int:
        if index < 0 or index >= self.size:
            return -1
        return self._get_node(index).val

    def addAtHead(self, val: int) -> None:
        self._insert_between(val, self.head, self.head.next)

    def addAtTail(self, val: int) -> None:
```

```
        self._insert_between(val, self.tail.prev, self.tail)

    def addAtIndex(self, index: int, val: int) -> None:
        if index < 0 or index > self.size:
            return

        if index == self.size:
            self._insert_between(val, self.tail.prev, self.tail)
        else:
            next_node = self._get_node(index)
            self._insert_between(val, next_node.prev, next_node)

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.size:
            return

        self._delete_node(self._get_node(index))
```

15. Key Takeaways

- The core pattern is linked-list design with sentinels.
- `size` makes invalid-index checks constant time.
- Dummy `head` and `tail` nodes remove most boundary special cases.
- A doubly linked list lets us traverse from the closer side.
- The interview invariant: sentinels always bound the real list, and every operation preserves correct `prev` and `next` links.